**PROJECT DOCUMENTATION**

**Title: AI Medical Assistant**

## 1. Introduction

This project presents an AI-powered medical assistant that enables users to query medical documents and receive accurate, context-based responses. The system is built using a **Retrieval-Augmented Generation (RAG)** architecture, combining document retrieval with large language model capabilities.

The application uses **AWS Bedrock** for both embedding generation and text generation, ensuring a fully cloud-based AI pipeline. A **FastAPI backend** handles document processing and querying, while a **Streamlit frontend** provides an interactive interface for users.

## 2. Objectives

- To develop a medical question-answering system using document-based retrieval
- To implement a RAG pipeline using AWS Bedrock
- To ensure responses are strictly grounded in medical documents
- To design a scalable backend using FastAPI
- To provide a user-friendly interface for querying medical data
- To integrate cloud-based embeddings and LLM services

## 3. Technologies and Libraries Used

### 3.1 Technologies

- Python: Core programming language
- FastAPI: Backend API framework
- Streamlit: Frontend interface
- AWS Bedrock: LLM and embedding services
- FAISS: Vector database for similarity search

### 3.2 Libraries

- boto3: AWS SDK for Bedrock integration
- langchain_core.embeddings: Custom embedding interface
- langchain_community.document_loaders: Document parsing
- langchain_community.vectorstores: FAISS integration
- python-dotenv: Environment variable management

- requests: API communication
- shutil, os: File handling

## 4. System Architecture

### 4.1 Workflow

PDF Upload → Document Loader → Embedding Generation (Bedrock) → Vector Storage (FAISS) → Query Input → Similarity Search → Context Retrieval → Prompt Construction → LLM (Bedrock) → Response

### 4.2 Component Description

**Frontend (Streamlit)**

- Allows users to upload medical PDF documents
- Provides input field for queries
- Displays conversation history

**Backend (FastAPI)**

- Handles file upload and indexing
- Manages vector storage and retrieval
- Processes user queries and generates responses

**Document Loader**

- Uses PyPDFLoader to extract text from PDF files

**Embedding Module**

- Custom implementation using AWS Bedrock embedding model
- Converts text into vector representations

**Vector Store (FAISS)**

- Stores embeddings locally
- Enables fast similarity-based retrieval

**Retriever**

- Retrieves top-k relevant document chunks for a query

**Prompt Engineering**

- Ensures strict constraints:
    - Answers must come only from provided context
    - No hallucination

    - Short and clear responses

**LLM Module (Bedrock)**

- Uses a configured Bedrock model for text generation
- Generates final answers based on retrieved context

**5. Features**

- Medical document-based question answering
- Retrieval-Augmented Generation pipeline
- Integration with AWS Bedrock for embeddings and LLM
- PDF upload and indexing
- Context-aware response generation
- Interactive Streamlit interface
- Source metadata returned with answers

**6. API Endpoints**

**POST /upload**

Uploads and indexes a medical PDF

**Input:**

- file (PDF)

**Output:**

- Confirmation message

**POST /ask**

Answers a query using indexed documents

**Input (query parameter):**

- query (string)

**Output:**

```
{
  "answer": "string",
  "sources": ["metadata"]
}
```

**Root Endpoint**

Returns system status

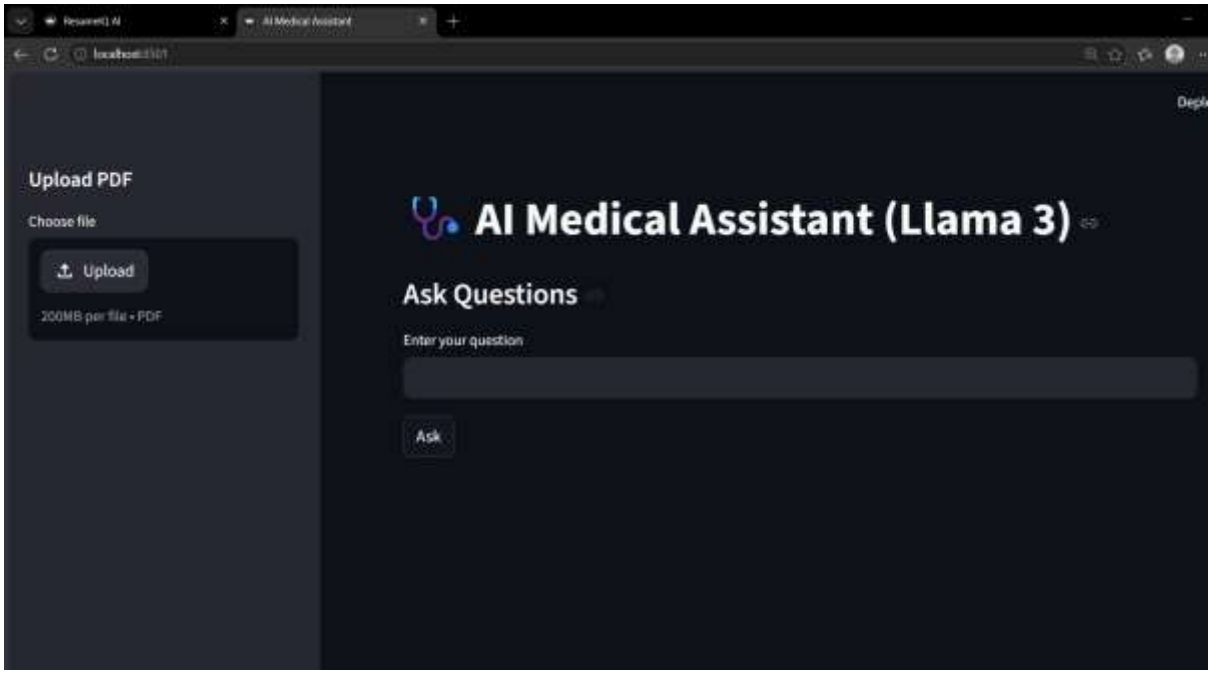
**7. Code Implementation**

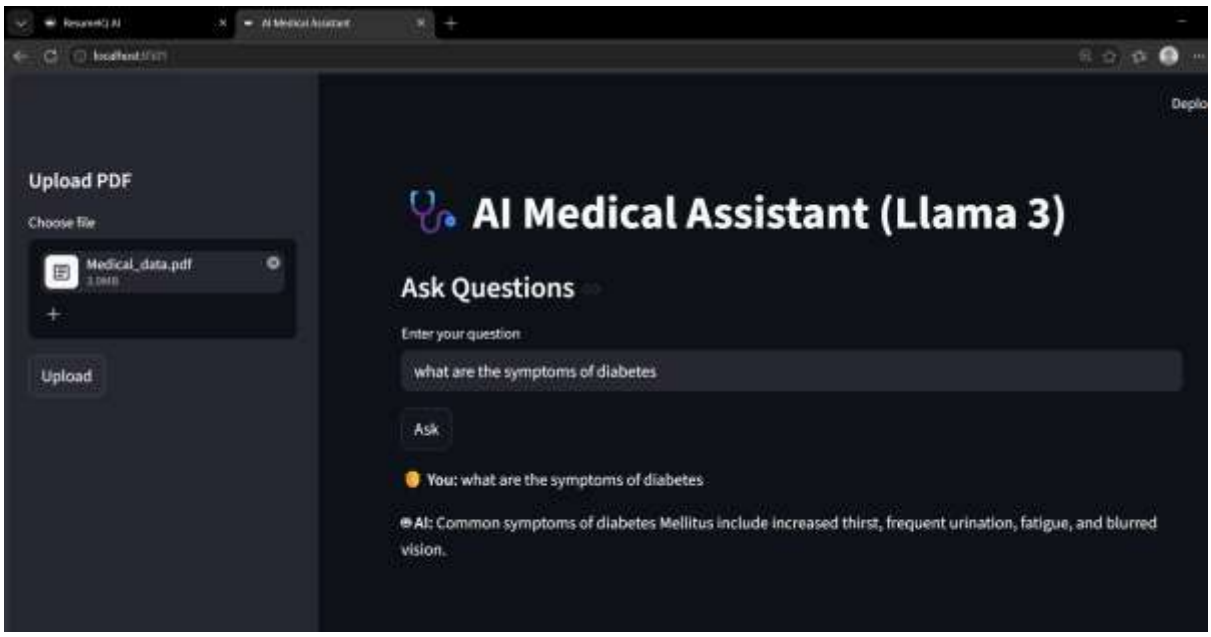
```
from app.vector_store import load_vector_store

from app.llm import BedrockLLM


def get_answer(query):

    db = load_vector_store()

    docs = db.similarity_search(query, k=5)


    context = "\n\n".join([doc.page_content for doc in docs])


    prompt = f"""

You are a medical AI assistant.


RULES:

- Answer ONLY from the provided context

- If answer is not in context, say "I don't know"

- Do NOT hallucinate

- Keep answer clear and short


Context:

{context}


Question:

{query}


Answer:

"""
```

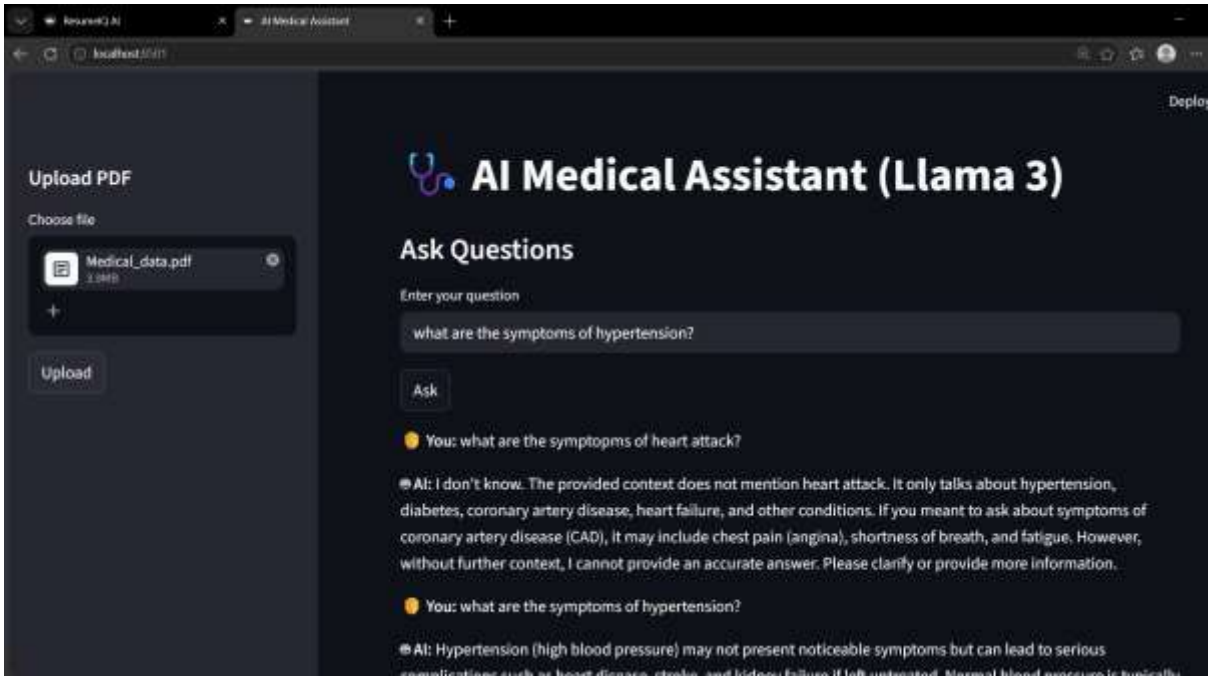
```
    llm = BedrockLLM()
    answer = llm.generate(prompt)

    return answer, docs

from fastapi import APIRouter, UploadFile, File
import shutil
import os

from app.document_loader import load_pdf
from app.vector_store import create_vector_store
from app.rag_pipeline import get_answer
from app.config import DATA_PATH

router = APIRouter()

@router.post("/upload")
async def upload_file(file: UploadFile = File(...)):
    file_path = os.path.join(DATA_PATH, file.filename)

    with open(file_path, "wb") as buffer:
        shutil.copyfileobj(file.file, buffer)

    docs = load_pdf(file_path)
    create_vector_store(docs)

    return {"message": "File uploaded and indexed successfully"}

@router.post("/ask")
async def ask_question(query: str):
```

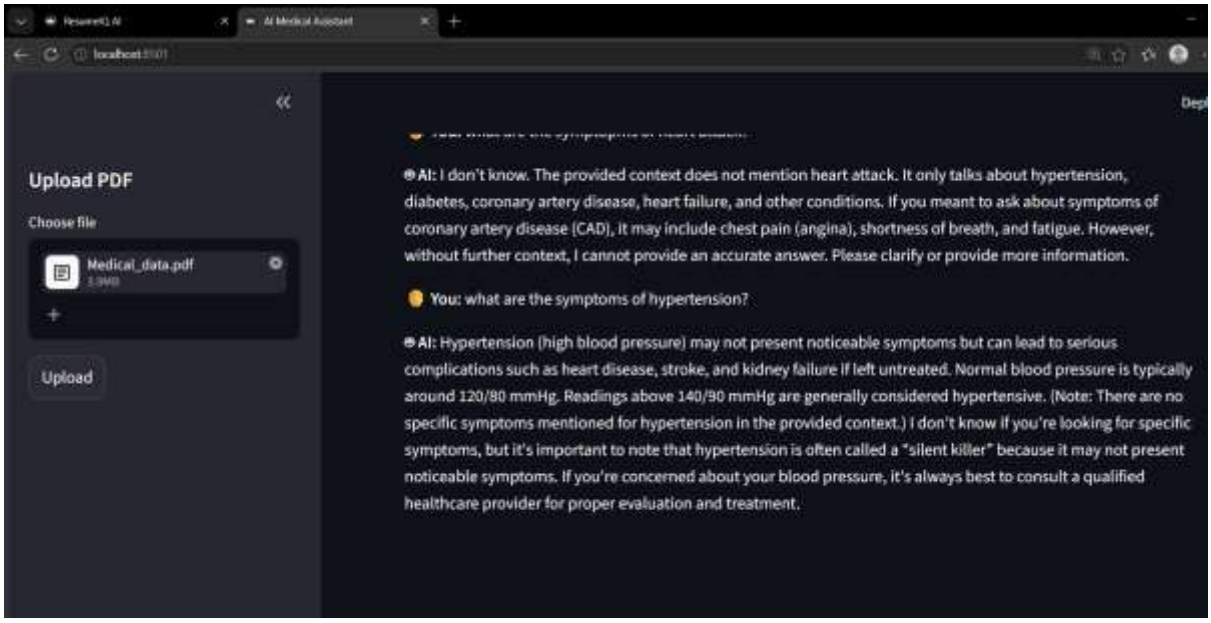
```
    answer, docs = get_answer(query)

    return {
        "answer": answer,
        "sources": [doc.metadata for doc in docs]
    }
```

**8. Output**






## 9. Conclusion

This project demonstrates the application of Retrieval-Augmented Generation in the medical domain using AWS Bedrock. By combining document retrieval with LLM-based generation, the system ensures accurate and context-bound responses. The modular architecture allows for future scalability and integration into advanced healthcare AI systems.